

🐍 Python Developer Roadmap — Beginner to Job Ready

1. Phase 0 — Understand the Basics

****Days 1–3****

- * What is programming?
- * What is Python?
- * How Python works
- * Install Python
- * Install/use VS Code
- * Terminal / Command Prompt
- * Create and run `.py` files
- * Basic debugging
- * How to read error messages

🎯 ****Goal:**** Run Python programs independently.

2. Phase 1 — Python Fundamentals

****Days 4–30****

2.1 Variables & Data Types

- * Variables
- * `int`
- * `float`
- * `str`
- * `bool`
- * `None`
- * Type conversion
- * `type()`

2.2 Operators

- * Arithmetic
- * Comparison
- * Logical
- * Assignment
- * Membership
- * Identity

2.3 Input & Output

- * `input()`
- * `print()`
- * f-strings

2.4 Conditions

- * `if`
- * `elif`
- * `else`
- * Nested conditions

2.5 Loops

- * `for`
- * `while`
- * `range()`
- * `break`
- * `continue`

2.6 Strings

- * Indexing
- * Slicing
- * String methods
- * Formatting

2.7 Collections

- * Lists
- * Tuples
- * Sets
- * Dictionaries

2.8 Functions

- * Creating functions
- * Parameters
- * Arguments
- * `return`
- * Scope
- * Default arguments
- * `\*args`

* `kwargs`

2.9 Files & Errors

* Reading files

* Writing files

* JSON

* CSV basics

* Exceptions

* `try`

* `except`

* `finally`

* `raise`



Project: Student Management System

3. Phase 2 — Strong Python

Days 31–60

3.1 OOP

* Classes

* Objects

* Constructors

* Methods

* Inheritance

* Encapsulation

* Polymorphism

* Abstraction

3.2 Advanced Python

* List comprehensions

* Dictionary comprehensions

* Lambda

* `map()`

* `filter()`

* `zip()`

* `enumerate()`

* Iterators

* Generators

- * Decorators

3.3 Python Environment

- * `pip`
- * Virtual environments
- * Packages
- * Modules
- * `requirements.txt`

 ****Projects:****

- * Expense Tracker
- * Bank Account System

4. Phase 3 — Problem Solving

****Days 61–75****

- * Big-O basics
- * Searching
- * Sorting
- * Recursion
- * Stack
- * Queue
- * Hashing
- * Basic data structures

 ****Practice:**** 3–5 problems every day.

****Important:**** Try solving the problem yourself before watching the solution.

5. Phase 4 — SQL & Databases

****Days 76–95****

5.1 Database Basics

- * What is a database?
- * What is SQL?

- * Tables
- * Rows
- * Columns
- * Primary key
- * Foreign key
- * CRUD

5.2 SQL

- * `SELECT`
- * `WHERE`
- * `ORDER BY`
- * `GROUP BY`
- * `HAVING`
- * Aggregate functions
- * JOINS
- * Subqueries
- * Constraints
- * Database relationships

5.3 PostgreSQL

- * PostgreSQL basics
- * Creating databases
- * Creating tables
- * Connecting to PostgreSQL

 ****Project:**** Employee Management System

6. Phase 5 — Git & GitHub

****Days 96–105****

- * What is Git?
- * What is GitHub?
- * Repository
- * Commit
- * Branch
- * Merge
- * Clone
- * Push
- * Pull

- * `.gitignore`
- * README

 ****Goal:**** Put all your projects on GitHub.

7. Phase 6 — Web Fundamentals

****Days 106–120****

7.1 Internet Basics

- * Internet
- * Client
- * Server
- * Browser
- * IP address
- * Domain
- * Port

7.2 Web Communication

- * HTTP
- * HTTPS
- * Request
- * Response
- * JSON
- * REST
- * API
- * Status codes

7.3 Basic Frontend Understanding

- * HTML
- * CSS

 ****Goal:**** Understand how web applications work before starting backend development.

8. Phase 7 — Python Backend

****Days 121–150****

8.1 FastAPI

- * What is backend?
- * FastAPI
- * Routes
- * Endpoints
- * GET
- * POST
- * PUT
- * PATCH
- * DELETE

8.2 API Parameters & Data

- * Path parameters
- * Query parameters
- * Request body
- * Response
- * Pydantic
- * Validation

8.3 API Documentation

- * Swagger
- * OpenAPI

 **Project:** Book Management API

9. Phase 8 — Database + Backend

****Days 151–165****

- * PostgreSQL with Python
- * SQLAlchemy
- * ORM
- * Models
- * CRUD
- * Relationships
- * Migrations
- * Database sessions

Project: Student Management REST API

10. Phase 9 — Authentication

****Days 166–180****

- * User registration
- * Login
- * Password hashing
- * JWT
- * Access tokens
- * Authentication
- * Authorization
- * Roles
- * Protected routes

 ****Goal:**** Add authentication to your API.

11. Phase 10 — Django

****Days 181–200****

11.1 Django

- * Django project
- * Django apps
- * URLs
- * Views
- * Models
- * Templates
- * Admin
- * Migrations
- * Django ORM
- * CRUD

11.2 Django REST Framework

- * Serializers
- * API views
- * ViewSets

- * Routers
- * Authentication
- * Permissions

12. Phase 11 — Professional Developer Skills

****Days 201–215****

12.1 Linux

- * Terminal
- * Files/directories
- * Permissions
- * Processes
- * Environment variables

12.2 Testing

- * Unit testing
- * `pytest`
- * API testing
- * Test cases

12.3 Docker

- * Images
- * Containers
- * Dockerfile
- * Docker Compose

12.4 Other Important Skills

- * `.env`
- * Logging
- * Debugging
- * API documentation

13. Phase 12 — Final Portfolio

****Days 216–240****

Project 1 — Employee Management API

Include:

- * Registration
- * Login
- * JWT
- * Roles
- * PostgreSQL
- * CRUD
- * Search
- * Filtering
- * Pagination
- * Validation
- * Error handling
- * Tests
- * Docker

Project 2 — Choose One

- * E-commerce API
- * Job Portal
- * Expense Management System

Include:

- * Authentication
- * Database
- * REST API
- * CRUD
- * Search
- * Filtering
- * Testing

 ****GitHub:**** Both projects should have proper README files and clear project explanations.

14. Phase 13 — Job Preparation

****Final 3–4 Weeks****

Python

- * 100 interview questions
- * 50 coding problems

SQL

- * 50 SQL problems
- * SQL interview questions

Backend

- * REST API
- * HTTP
- * Authentication
- * JWT
- * Database
- * FastAPI
- * Django

Projects

Be able to explain:

- * What did you build?
- * Why did you choose Python?
- * How does the API work?
- * How is the database connected?
- * How does authentication work?
- * What problems did you face?
- * How did you solve them?

 ****Final Goal:**** Be able to ****code, build projects, explain your projects, use GitHub, and confidently answer beginner-to-intermediate Python/backend interview questions.****